1 2 3 4 5 6 7 8

Tier 1 — Building Blocks for Skills

The eight primitives every AI skill is assembled from — and the rules that keep them safe, testable and reusable

AI Specialisation · Fresher cohort · FIL Gurgaon

Skills Engineering Strategy: Building Blocks → Skills OS → Platform → Skills-as-a-Service

How today works



Concept
Short explainer, one idea per slide



Discuss
Think-pair-share, 3–5 minutes



Check
Quick quiz, answers on the next slide



Lab
Hands-on build with clear 'done when' criteria



Badge
Earned for every lab you complete

Agenda (≈ 6.5 hours)

Part A	What a skill is, and why primitives	30 min
Part B	Instruction + Trigger · Lab 1	70 min
Part C	Knowledge → Evals · Labs 2 & 3	110 min
Part D	Design rules · Lab 4	60 min
Part E	Skill taxonomy	30 min
Part F	Capstone Lab 5 + exit ticket	75 min

By the end of today you can…

Name the eight primitives

and say what each one is for in one sentence

Write a SKILL.md

with role, procedure, decision rules, output contract and examples

Tune a trigger

so the skill loads when it should — and only then

Build a typed Action

with a JSON Schema contract and errors the model can act on

Declare permissions

using least privilege, and pass a security gate

Classify skills

as atomic, composite, policy or agent profile

Part A · What is a skill?

And why we standardise its parts

Warm-up: your first week at FIL team

DISCUSS

Imagine you are joining the Accounts Payable team tomorrow. What would you need on day one to do the job well, without asking someone every five minutes?

A procedure

Step-by-step: how we process an invoice

When to use it

Which requests are yours, which are not

Reference docs

Policies, vendor list, glossary

Tools

Calculator, ERP screens, scripts

Templates

The approved email and report formats

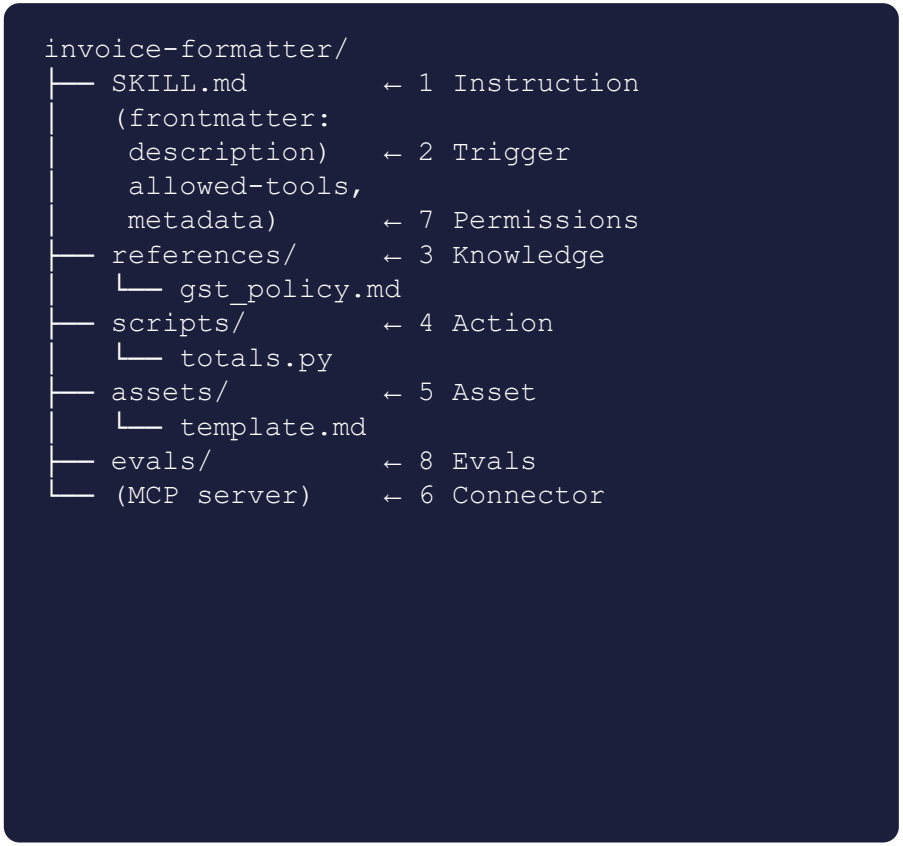
Access + limits

What you may read, change, approve

A skill = an onboarding pack for an AI agent

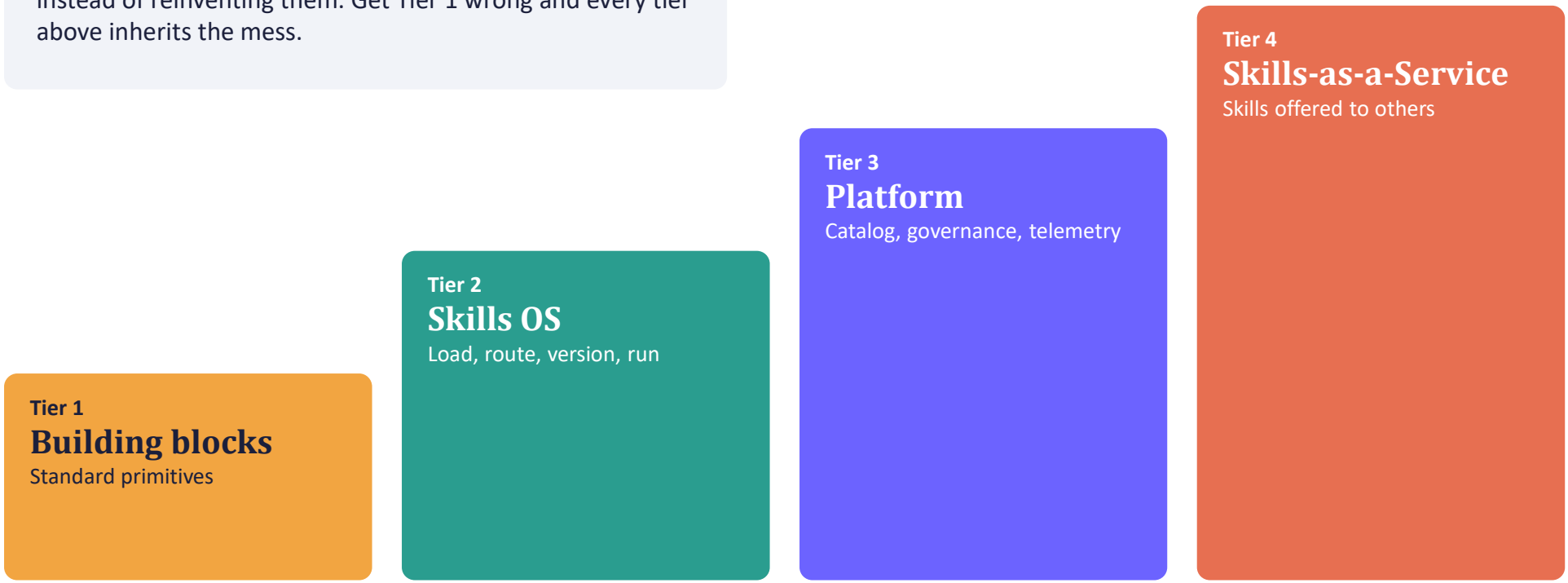
A skill is a folder an agent can discover, load on demand and follow, so it does one kind of job the way your company wants it done.

- Portable: a plain folder, versioned in git
- Lazy: only a one-line description is always in context
- Composable: skills call tools and other skills
- Governed: declares what it is allowed to touch
- Testable: ships with evals, like code ships with unit tests



Where Tier 1 sits in the strategy

You are here. Every higher tier composes these blocks instead of reinventing them. Get Tier 1 wrong and every tier above inherits the mess.



Why standardise? Compose, don't reinvent

Without a standard

- Every team writes prompts in its own format
- Tools return free text; the agent guesses
- Nobody knows what a skill can access
- Model upgrade? Nobody knows what broke
- Same 'format a date' logic copied 40 times

With Tier 1 primitives

- One SKILL.md shape everyone can review
- Typed JSON Schema contracts for every tool
- A permission manifest the security gate checks
- Evals re-run automatically on every model upgrade
- Atomic skills reused by composite skills

The eight primitives at a glance

1

Instruction

SKILL.md body: role, procedure, decision rules, output contract, examples

2

Trigger

The description line the agent reads before loading

3

Knowledge

references/: docs, policies, glossaries, schemas, loaded on demand

4

Action

Deterministic code in scripts/ or an MCP tool with a JSON Schema contract

5

Asset

Templates, brand files, sample outputs in assets/

6

Connector

An MCP server reaching a system of record, with auth

7

Permission manifest

Declared tools, data scopes, network egress, side effects

8

Evals

Tests for both failure surfaces: trigger and execution

What it knows

What it can do

What keeps it safe

Part B · Instruction and Trigger

Primitives 1 and 2: what the agent does, and when it decides to do it

1 · Instruction: the SKILL.md body

1

The written procedure the agent follows once the skill is loaded. Owned by the domain author: the person who actually knows how the job is done.



Role

Who the agent is acting as. "You are an AP analyst at FIL."



Procedure

Numbered steps, in order. One action per step.



Decision rules

If-then rules for the tricky cases. "If GSTIN is missing, stop and ask."



Output contract

Exact shape of the result: fields, format, order.



Examples

1–3 input → output pairs that show 'good'.

What a good SKILL.md looks like

```
---
name: invoice-formatter
description: Use when the user asks to summarise
  or format a vendor invoice into the FIL invoice
  summary. Do not use for purchase orders,
  payments or tax filing.
---
## Role
You are an AP analyst at FIL.
## Procedure
1. Extract vendor, GSTIN, date, line items.
2. Run scripts/totals.py for all arithmetic.
3. Fill assets/summary_template.md.
## Decision rules
- If GSTIN is missing: stop, ask the user.
- If totals differ by > ₹1: flag MISMATCH.
## Output contract
Markdown table: Field | Value, then Status.
## Examples  → see references/examples.md
```

1. **Frontmatter**
   name + description live between the --- lines

2. **Short procedure**
   Steps point to scripts/ and assets/ instead of repeating them

3. **Rules are testable**
   Each rule could become an eval case

4. **Contract is exact**
   Another program could parse this output

Keep it lean: the instruction budget

< 5,000	< 500	≈ 100
tokens in the SKILL.md body	lines recommended	tokens per description, always loaded

Why the budget matters

- Every token you load competes with the user's data for the model's attention
- Long instructions hide the important rule in the middle, where models follow it least reliably
- Big detail belongs in references/ and is loaded only when a step needs it

2 · Trigger: the line read before loading

2 The description in the frontmatter. It is the only part of your skill the agent sees when deciding whether to load it.

Progressive disclosure: three levels

Level 1 · Start-up

Metadata

name + description of every installed skill. Always in context.

→

Level 2 · On match

SKILL.md body

Loaded only when the trigger matches the request.

→

Level 3 · On demand

references/, scripts/, assets/

Opened only when a step in the procedure needs them.

Quality is owned by the compiler + trigger evals: the description is tested, not just written.

Writing a trigger: 'Use when… Do not use for…'

Weak

```
description: Helps with invoices
  and finance stuff.
```

Strong

```
description: Use when the user asks to
  format a vendor invoice into the FIL
  summary. Do not use for POs or tax.
```

Checklist for a strong trigger

- **Names the task** using words users actually type
- **Says 'Use when…'** the positive cases
- **Says 'Do not use for…'** the near-misses it should ignore
- **Is specific** to FIL, not generic advice

Check 1 · Instruction and Trigger

CHECK

Q1 **Which part of a skill does the agent read BEFORE deciding to load it?**

A) The SKILL.md body
B) The description line
C) The scripts/ folder
D) The assets/ folder

Q2 **Your SKILL.md has grown to 1,200 lines. What is the best fix?**

A) Keep it; context windows are big
B) Move detailed policies and examples into references/
C) Delete the decision rules
D) Turn the whole skill into one Python script

Q3 **Which is NOT one of the five parts of an Instruction?**

A) Output contract
B) Decision rules
C) Database password
D) Examples

Check 1 · Answers

ANSWERS

Q1 → B · The description line

Only Level 1 metadata (name + description) is always in context. The body loads after a match.

Q2 → B · Move detail into references/

Stay under ~500 lines. Detail moves to Level 3 and is loaded only when a step needs it.

Q3 → C · Database password

Credentials never live in instructions. Access comes from connectors and the permission manifest.

Lab 1 · Write your first SKILL.md

LAB

25 minutes · pairs · skill: invoice-summary-formatter

1. Create a folder invoice-summary/ with an empty SKILL.md inside
2. Write the frontmatter: name, and a description with 'Use when…' and 'Do not use for…'
3. Write the body: Role, Procedure (numbered), Decision rules (if-then), Output contract
4. Add two input → output examples (use the sample invoices handed out)
5. Count lines and estimate tokens: stay under 500 lines
6. Paste SKILL.md as the system prompt via the shared llm_client.py and run all 3 sample invoices

Done when…

- Frontmatter has both clauses
- All 5-instruction parts are present
- Output matches your contract on 3 of 3 invoices
- A neighbour pair can follow it without asking you

Badge unlocked
Instruction Author

Lab 1 · Starter skeleton

LAB

```
---
name: invoice-summary-formatter
description: Use when … Do not use for …
---
## Role
You are …

## Procedure
1. …
2. …
3. …

## Decision rules
- If … then …

## Output contract
| Field | Value |
|---|---|
Status: OK | MISMATCH | NEEDS_INFO

## Examples
Input: …   Output: …
```

Tips

- One action per procedure step
- Name the fields exactly
- Use a fixed set of status values

Avoid

- "Be helpful and accurate" (not testable)
- Doing maths in the prompt
- Secrets or URLs with tokens

Part C · Primitives 3 to 8

Knowledge, Action, Asset, Connector, Permissions, Evals: what the skill knows, what it can do, and what keeps it safe

3 · Knowledge: loaded only when needed

3

Docs, policies, glossaries and schemas the procedure points to. Owned by the knowledge owner (for example, the finance policy team).



Static: references/

- Files inside the skill folder
- Changes slowly: policy, glossary, schema
- Versioned with the skill in git
- Example: references/gst_policy.md



Live: MCP Resources

- Read from a server at run time
- Changes often: vendor master, exchange rates
- Always current, needs a connector
- Example: vendors://active-list

4 · Action: deterministic code with a contract

4 When an answer must be exact, the model calls code instead of guessing. A script in scripts/ or an MCP tool, described by a JSON Schema 2020-12 contract.

```
{
 "$schema": "https://json-schema.org/
             draft/2020-12/schema",
 "title": "compute_invoice_totals",
 "type": "object",
 "required": ["lines", "gst_rate"],
 "properties": {
  "lines": {"type": "array", "minItems": 1},
  "gst_rate": {"enum": [0, 5, 12, 18, 28]}
 },
 "additionalProperties": false
}
```

Use for

Calculations, file formats, API calls

The contract

Tells the model exactly what to send, and rejects anything else

Owner

Engineer or plugin vendor

Errors teach the model

A tool result is read by the model, so an error message is an instruction for the next attempt. Make it actionable.

Opaque failure

```
Error 500

Traceback (most recent…
KeyError: 'gst_rate'
```

The agent cannot tell what to fix, so it retries the same call or gives up.

Actionable error

```
{"error": "gst_rate missing",
 "fix": "Send gst_rate as one of
   0, 5, 12, 18, 28",
 "retryable": true}
```

Says what went wrong, how to fix it, and whether retrying can help. The agent self-corrects.

5 · Asset: what 'good' looks like

5

Templates, brand files and sample outputs in assets/. The agent copies the shape instead of inventing one. Owned by the author.

Templates

summary_template.md, report.docx, email_reply.txt

Brand files

Logo, colour codes, approved fonts, footer text

Sample outputs

A 'gold' example the result should look like

Knowledge vs Asset? Knowledge is read to decide (the GST policy). An asset is used to produce (the invoice template). If the agent fills it in or copies it, it is an asset.

6 · Connector: reaching a system of record

6

An MCP server that connects the agent to a real system (ERP, CRM, HR) with proper authentication. Owned by the connector vendor.



Why on-behalf-of matters: the ERP sees the request as Priya, with Priya's permissions, not as a shared super-account. Audit logs stay truthful and the agent can never see more than the user could.

Standard: MCP (2026-07-28 revision) + OAuth; RFC 8693 token exchange for on-behalf-of.

7 · Permission manifest: declare before you touch

7

A declaration of every tool, data scope, network destination and side effect. The security gate reads it before the skill ships.

```
---
name: invoice-summary-formatter
description: Use when …
allowed-tools: Read Bash(python:*)
metadata:
  fil.permissions:
    data_scopes: [ap.invoices:read]
    network_egress: none
    writes: [./output/]
    side_effects: none
    credentials: none
---
```

Tools · allowed-tools in the spec

Data scopes · Which records, read or write

Network egress · Which hosts, or none

Side effects · Emails, payments, tickets

8 · Evals: two ways a skill can fail

8

Every primitive set ships tests for both failure surfaces, and they re-run on every model upgrade.

Should trigger · did trigger

Then check: was the output correct?

Should trigger · did NOT

Missed trigger. Fix the description.

Should NOT · did trigger

False trigger. Add 'Do not use for…'.

Triggered · output wrong

Execution failure. Fix instruction, script or knowledge.

Two eval sets

- **Trigger evals**
- Prompts that should and should not load the skill
- **Quality evals**
- Inputs with expected outputs, checked against the contract

Who owns the quality of each primitive?

#	Primitive	Standard / reference	Owner of quality
1	**Instruction**	Agent Skills spec (<5,000 tokens, <500 lines)	Domain author
2	**Trigger**	Agent Skills progressive disclosure	Compiler + trigger evals
3	**Knowledge**	Agent Skills; MCP Resources for live data	Knowledge owner
4	**Action**	MCP Tools; JSON Schema 2020-12	Engineer / plugin vendor
5	**Asset**	Agent Skills	Author
6	**Connector**	MCP 2026-07-28 + OAuth; RFC 8693	Connector vendor
7	**Permission manifest**	allowed-tools + metadata extension	Security gate
8	**Evals**	Trigger + quality eval suites	Skill owner + CI

Check 2 · Which primitive is it?

CHECK

Q1 **The skill must read today's approved vendor list from the ERP.**

A) Knowledge (references/) B) Connector + MCP Resource C) Asset D) Instruction

Q2 **GST must be calculated exactly on 40 line items.**

A) Instruction B) Asset C) Action D) Trigger

Q3 **The approved FIL invoice-summary layout.**

A) Asset B) Knowledge C) Permission manifest D) Connector

Q4 **"network_egress: none, writes: [./output/]"**

A) Trigger B) Evals C) Action D) Permission manifest

Check 2 · Answers

ANSWERS

Q1 → B · Connector + MCP Resource

Live data that changes daily comes from a system of record through an authenticated MCP server.

Q2 → C · Action

Arithmetic needs determinism, so it lives in scripts/ or an MCP tool with a JSON Schema contract.

Q3 → A · Asset

The agent uses it to produce output, so it is an asset, not reference knowledge.

Q4 → D · Permission manifest

Declared scopes and side effects, checked by the security gate before release.

Lab 2 · Build an Action with a contract

LAB

30 minutes · pairs · Python 3.11+, pip install jsonschema

1. Create scripts/totals_schema.json using JSON Schema 2020-12 (lines, gst_rate, additionalProperties: false)
2. Write scripts/totals.py: read JSON from stdin, validate with Draft202012Validator
3. Compute subtotal, GST and total using Decimal, rounded to 2 places
4. On any error, print {"error", "fix", "retryable"} and exit with code 2
5. Test 4 inputs: valid, missing gst_rate, gst_rate 18.5, empty lines
6. Update SKILL.md step 2 to call the script instead of doing maths

Done when…

- Valid input gives exact totals
- All 3 bad inputs return an actionable 'fix'
- No stack trace reaches the model
- SKILL.md references the script

Badge unlocked
Action Engineer

Lab 2 · Five edits to pass all 13 tests

Each step: open this file, find this text, replace it, run pytest, check the result.

Before you start

1. **Open a terminal in the Lab 2 student folder**

   The folder that contains tests/ inputs/ invoice-summary-formatter/

2. **Install once**

   ```
   pip install -r tools/requirements.txt
   ```

3. **Run the tests**

   ```
   pytest -q          (if not found: python -m pytest -q)
   ```

4. **You should see: 13 failed**

   That is correct. The starter is unfinished on purpose.

5. **After EVERY step: [OK] or what to fix**

   ```
   python tools/check_lab2.py
   ```

Step	File	Edit	Check shows
1	`scripts/totals_schema.json`	Replace the whole file	[OK] Step 1
2	`scripts/totals.py`	Replace the TODO 2 comments	[OK] Step 2
3	`scripts/totals.py`	Add explain(), replace 4 lines	[OK] Step 3
4	`scripts/totals.py`	Replace 1 line	13 passed
5	`SKILL.md`	Replace procedure step 5	check_skill PASS

Step 1 · Replace the schema

File: `invoice-summary-formatter/scripts/totals_schema.json`

1. **Select everything in the file**

   Ctrl+A, then Delete

2. **Paste the new schema**

   Exactly as shown on the right →

3. **Save (Ctrl+S), then run**

   `python tools/check_lab2.py`

[OK] Step 1

pytest still shows 13 failed: that is normal until step 2.

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "compute_invoice_totals",
  "type": "object",
  "required": ["lines", "gst_rate"],
  "properties": {
    "lines": {
      "type": "array",
      "minItems": 1,
      "items": {
        "type": "object",
        "required": ["qty", "price"],
        "properties": {
          "description": {"type": "string"},
          "qty": {"type": "number", "exclusiveMinimum": 0},
          "price": {"type": "number", "minimum": 0}
        },
        "additionalProperties": false
      }
    },
    "gst_rate": {"enum": [0, 5, 12, 18, 28]},
    "stated_total": {"type": "number", "minimum": 0}
  },
  "additionalProperties": false
}
```

Step 2 · Add the calculation

File: `invoice-summary-formatter/scripts/totals.py`

1 **Find the last lines of the file**

the 6 comment lines that start with
TODO 2: compute …

2 **Delete those 6 lines**

and paste the code on the right in their place, at the left margin (no spaces in front).

3 **Save, then run**

`python tools/check_lab2.py`

[OK] Step 2

pytest -q now: 7 failed, 6 passed.

```
CENT = Decimal("0.01")

def money(x):
    """Exact money: Decimal made from text, rounded half-up to 2 places."""
    return Decimal(str(x)).quantize(CENT, rounding=ROUND_HALF_UP)

subtotal = Decimal("0")
for line in data["lines"]:
    amount = Decimal(str(line["qty"])) * Decimal(str(line["price"]))
    subtotal += money(amount)
gst = money(subtotal * Decimal(str(data["gst_rate"])) / 100)
total = subtotal + gst

result = {"subtotal": f"{subtotal:.2f}", "gst": f"{gst:.2f}",
          "total": f"{total:.2f}", "stated_total": None,
          "difference": None, "status": "OK"}

if "stated_total" in data:
    stated = money(data["stated_total"])
    difference = stated - total
    result["stated_total"] = f"{stated:.2f}"
    result["difference"] = f"{difference:.2f}"
    if abs(difference) > Decimal("1.00"):
        result["status"] = "MISMATCH"

print(json.dumps(result))
```

Step 3a · Add the explain() function

File: `invoice-summary-formatter/scripts/totals.py`

1 **Find the fail() function**

```
def fail(msg, fix):
    print(…)
    sys.exit(2)
```

2 **Paste the WHOLE block on the right**

on a new line just below sys.exit(2), at the left margin.

3 **Save, then run**

```
python tools/check_lab2.py
```

Step 3b: … not used yet

Correct. explain() is used in step 3b.

```
def explain(e):
    """Turn one schema error into a fix sentence the model can act on."""
    rates = "0, 5, 12, 18, 28"
    if "gst_rate" in e.message or "gst_rate" in e.json_path:
        return f"Send gst_rate as one of {rates} (a number, e.g. 18)."
    if e.validator == "required":
        return "Add the missing field: " + e.message.split("'")[1]
    if e.validator == "minItems":
        return "Send at least one line item with qty and price."
    if e.validator == "additionalProperties":
        return "Remove the unknown field. Allowed: lines, gst_rate, stated_total."
    if e.validator == "exclusiveMinimum":
        return "qty must be greater than 0."
    if e.validator == "minimum":
        return "price and stated_total must be 0 or more."
    return "Fix the input so it matches scripts/totals_schema.json."
```

Step 3b · Use explain() for errors

File: `invoice-summary-formatter/scripts/totals.py`

Find these 4 lines:

```
errs = sorted(V.iter_errors(data), key=str)
if errs:
    e = errs[0]
    fail(e.message, "Check field: " + e.json_path)    # TODO 3: make the fix specific per error type
```

Replace them with:

```
errs = sorted(V.iter_errors(data), key=lambda e: len(e.path))
if errs:
    e = errs[0]
    fail(e.message, explain(e))
```

Save, then run: python tools/check_lab2.py

[OK] Step 3

pytest -q now: 1 failed, 12 passed.

Step 4 · Handle input that is not JSON

File: `invoice-summary-formatter/scripts/totals.py`

Find this 1 line:

```
data = json.load(sys.stdin)          # TODO 4: invalid JSON must also return an actionable error
```

Replace it with:

```
try:
    data = json.load(sys.stdin)
except json.JSONDecodeError:
    fail("input is not valid JSON",
         "Send one JSON object, e.g. "
         '{"lines": [{"qty": 2, "price": 500}], "gst_rate": 18}')
```

Save, then run: python tools/check_lab2.py
then: pytest -q

13 passed

All tests pass. The script is done.

Step 5 · Tell the skill to use the script

File: `invoice-summary-formatter/SKILL.md`

In ## Procedure, find step 5 (4 lines):

```
5. Do the arithmetic visibly, never in your head, and never trust the printed totals:
   (a) add the line amounts one by one and write the sum in the "Line items sum" row, e.g. …;
   (b) add subtotal + GST amount and write it in the "Calculated total" row, e.g. …
   Add rupees and paise column by column. Only after writing both rows, compare them with the printed Subtotal and Grand
total.
```

Replace it with:

```
5. Do NOT do any arithmetic yourself. Put the invoice into JSON, for example
   `{"lines": [{"qty": 20, "price": 280}], "gst_rate": 18, "stated_total": 14750}`,
   and run `python scripts/totals.py` with that JSON as input. Write its "subtotal" in the
   "Line items sum" row and its "total" in the "Calculated total" row. If it returns an
   "error", follow its "fix" and run it again.
```

Save, then run: `python tools/check_skill.py invoice-summary-formatter` → RESULT: PASS

If something does not match

First run python tools/check_lab2.py : it names the step and the line to fix.

You see	Do this
6 failed, 7 passed: "errors must exit with code 2"	Step 1 not saved: totals_schema.json is still the old file. Redo step 1, Ctrl+S
NameError: name '…' is not defined	Part of a pasted block is missing. Delete it and paste the whole block again
"printed nothing" after step 2	Save the file. Check you edited invoice-summary-formatter/scripts/totals.py
Every test fails with JSONDecodeError	totals_schema.json has a typo: compare with step 1 (commas, braces)
IndentationError	Pasted code must start at the left margin, not indented
Still "Check field: …" after step 3	You added explain() but did not replace the 4 lines (step 3b)
ModuleNotFoundError: jsonschema	pip install -r tools/requirements.txt
collected 0 items	Run pytest from the Lab 2 student folder
PytestDeprecationWarning … asyncio (Anaconda)	Harmless warning from another plugin. Ignore it



Done when: pytest shows 13 passed and check_skill shows PASS.

Lab 2 · Starter code

LAB

```
import json, sys
from decimal import Decimal, ROUND_HALF_UP
from jsonschema import Draft202012Validator

SCHEMA = json.load(open("scripts/totals_schema.json"))
V = Draft202012Validator(SCHEMA)

def fail(msg, fix):
    print(json.dumps({"error": msg, "fix": fix,
                      "retryable": True}))
    sys.exit(2)

data = json.load(sys.stdin)
errs = sorted(V.iter_errors(data), key=str)
if errs:
    e = errs[0]
    fail(e.message, "Check field: " + e.json_path)

# TODO: subtotal, gst, total with Decimal
# TODO: print JSON result
```

Try it

```
echo '{"lines":[{"qty":2,
"price":500}],"gst_rate":18}'
| python scripts/totals.py
```

Expected: subtotal 1000.00, gst 180.00, total 1180.00

Improve the generic 'fix' text so it names the allowed values.

Lab 3 · Trigger evals: does it load at the right time?

LAB

25 minutes · pairs · uses the shared llm_client.py

1. Create evals/triggers.jsonl with 10 prompts: 5 should trigger, 5 near-misses that should not
2. Give the model 4 skill descriptions (yours + 3 provided decoys)
3. Ask: 'Which skill should be loaded? Reply with its name or NONE'
4. Score each prompt: correct trigger, missed trigger, or false trigger
5. Rewrite your description and re-run until at least 9 of 10 are correct
6. Record before and after scores on the class board

Done when…

- File has 5 positive + 5 near-miss prompts
- Score ≥ 9/10 on the final run
- You can explain which words fixed the false triggers
- Before/after score posted

Badge unlocked
Trigger Tuner

Lab 3 · Load at the right time

Write 10 test prompts, measure, fix the description, measure again: from 5/10 to 10/10.

Before you start

1. **Open a terminal in the Lab 3 student folder**

   The folder that contains skills/ evals/ tools/

2. **Install once**

   ```
   pip install -r tools/requirements.txt
   ```

3. **Copy your .env from Lab 1 into this folder**

   No credentials? Put LLM_MODE=mock in .env (works offline)

4. **Check the connection**

   ```
   python tools/check_setup.py
   ```

5. **After EVERY edit: [OK] or what to fix**

   ```
   python tools/check_lab3.py
   ```

Step	What you do	You should see
1	Create evals/triggers.jsonl	[OK] Step 1
2	Baseline run	Score 5/10
3	Read the result	5 missed triggers
4a	Add 'Use when ...'	Score 9/10
4b	Add 'Do not use ...'	Score 10/10
5	Final run x3, post score	30/30

Step 1 · Create the test prompts

File: `evals/triggers.jsonl`

1 Copy the template to a new file (pick your terminal):

```
bash / Git Bash :  cp evals/triggers_template.jsonl evals/triggers.jsonl
Windows cmd     :  copy evals\triggers_template.jsonl evals\triggers.jsonl
```

2 Open evals/triggers.jsonl. It has 3 lines. Paste these 7 lines at the end (one prompt per line):

```
{"prompt": "Check if the totals on this vendor invoice add up", "expected": "invoice-summary-formatter"}
{"prompt": "Format this tax invoice from NetLink into our AP summary", "expected": "invoice-summary-formatter"}
{"prompt": "Vendor sent invoice GCC-1187, please put it in the standard invoice summary", "expected": "invoice-summary-formatter"}
{"prompt": "Can you review this supplier bill and flag anything missing before I approve it?", "expected": "invoice-summary-formatter"}
{"prompt": "Schedule payment for invoice SOS/2026/0412 next Friday", "expected": "payment-scheduler"}
{"prompt": "What GST rate applies to catering services?", "expected": "gst-faq-answerer"}
{"prompt": "Create an invoice to bill our client for consulting work", "expected": "NONE"}
```

3 Save, then run: `python tools/check_lab3.py` → [OK] Step 1 triggers.jsonl has 10 prompts

Format of every line: {"prompt": "what a user types", "expected": "skill that should load, or NONE"}

Step 2 · Measure the starting point

1 Run the trigger eval (one line):

```
python tools/trigger_eval.py --skills skills --evals evals/triggers.jsonl --target invoice-summary-formatter
```

2 You should see (end of the output):

```
XX missed trigger   expected=invoice-summary-formatter  got=NONE
XX missed trigger   expected=invoice-summary-formatter  got=payment-scheduler
…
Score: 5/10
Missed triggers: 5   False triggers: 0
```

Before: 5/10

Write it on the class board.

Why so low? The starter description is only "Helps with invoices and finance stuff." The model cannot tell when this skill is the right one.

Step 3 · Read the result

Line starts with	Meaning	What to change
`OK correct`	The right skill loaded (or none, when none should)	**Nothing**
`XX missed trigger`	Your skill SHOULD have loaded but did not	**Add the user's words to 'Use when …' (step 4a)**
`XX false trigger`	Your skill loaded when it should NOT	**Name that case in 'Do not use …' (step 4b)**

Your baseline: 5 missed triggers. Look at the words in those 5 prompts:

```
Here's the BILL from Shakti … can you SUMMARISE it for approval?
CHECK if the TOTALS on this VENDOR INVOICE ADD UP
FORMAT this tax INVOICE from NetLink into our AP SUMMARY
… put it in the standard INVOICE SUMMARY
Can you REVIEW this SUPPLIER BILL …
```

Step 4a · Add "Use when ..."

File: `skills/invoice-summary-formatter/SKILL.md`

Find line 3:

```
description: Helps with invoices and finance stuff.
```

Replace it with (this is ONE line in the file):

```
description: Use when the user shares a vendor or supplier invoice or bill they received and asks to summarise,
check, review or format it into the FIL AP invoice summary, or to check whether its totals add up.
```

Save, then run: python tools/check_lab3.py → [OK] 4a, [FIX] 4b (correct for now)

Then run the trigger eval again (same command as step 2).

Score: 9/10
Missed triggers: 0 False triggers: 1

The 1 false trigger: "Create an invoice to bill our client for consulting work". It also says invoice and bill, but it is an invoice WE send. Step 4b fixes it.

Step 4b · Add "Do not use ..."

File: `skills/invoice-summary-formatter/SKILL.md`

On the same description line, put the cursor after ...totals add up. type one space and paste:

```
Do not use to create or raise an invoice for our own clients or customers, for purchase orders, for scheduling or making payments, for general GST rate questions, or for filing GST returns.
```

The finished line 3 (still ONE line):

```
description: Use when the user shares a vendor or supplier invoice or bill they received and asks to summarise, check, review or format it into the FIL AP invoice summary, or to check whether its totals add up. Do not use to create or raise an invoice for our own clients or customers, for purchase orders, for scheduling or making payments, for general GST rate questions, or for filing GST returns.
```

Save, then run: `python tools/check_lab3.py` → All edits done

Score: 10/10

Run the trigger eval again: no XX lines left.

Step 5 · Final run and post your score

1 Run 3 times per prompt (real models can vary between runs):

```
python tools/trigger_eval.py --skills skills --evals evals/triggers.jsonl --target invoice-summary-formatter --runs 3
```

2 You should see:

```
Score: 30/30
Missed triggers: 0    False triggers: 0
```

Target: 27/30 or more

With the real model, small differences are normal.

3 Write on the class board: before 5/10 → after 10/10 (30/30)



Done when: 10 prompts in triggers.jsonl, final score 27/30 or more, and you can say which words fixed the false trigger.

If something does not match

First run python tools/check_lab3.py : it names the step and the line to fix.

You see	Do this
JSONDecodeError … triggers.jsonl	A line is broken. check_lab3 shows the line number: fix quotes or commas
Target 'invoice-summary-formatter' not found	Run from the Lab 3 student folder; do not rename the skill folder
Score did not change after editing SKILL.md	File not saved, or you edited a different SKILL.md
check_skill says the name or description is broken	The description must stay on ONE line, starting with description:
Missing setting 'FIL_CLIENT_ID'	Copy your .env into this folder, or use LLM_MODE=mock
Real model: score changes between runs	Normal. Use --runs 3 and aim for 27/30 or more
A different prompt fails for you	Read its XX line: missed → add words to 'Use when', false → add to 'Do not use'

Part D · Five design rules for Tier 1

The habits that keep every skill simple, safe and upgrade-proof

The five rules on one page

1. Instruction-only first	Add scripts only where determinism matters
2. Two failure surfaces	Test triggering and execution
3. Errors teach the model	Return actionable error text
4. Least privilege by default	No network, write or credentials unless declared
5. Typed skill classes	Capability uplift vs encoded preference

Rule 1 · Instruction-only first

Start with words. Add code only when the answer must be exactly right every time.

Is it a calculation? → Yes → script / tool

Is it a strict file format? → Yes → script / tool

Is it an API call? → Yes → script / tool

All 'no'? Keep it instruction-only.

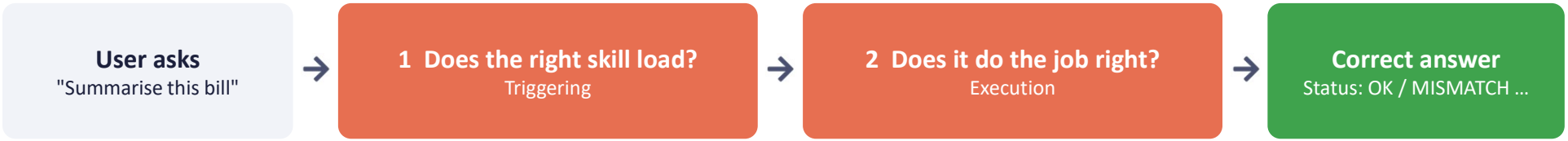


Why words first?

- Faster to write and review
- No code to maintain or secure
- Domain experts can edit it
- OpenAI's skill-creator defaults to instruction-only for the same reason

Rule 2 · Two failure surfaces

A skill can go wrong in two different places. Each place needs its own test.



Failure 1 · It does not load (or loads wrongly)

"Here's the bill from Shakti…" → invoice skill did NOT load (missed trigger)
"Create an invoice for our client" → it DID load (false trigger)

Test with: Trigger evals (Lab 3)
Fix: the description

Failure 2 · It loads, but the answer is wrong

Invoice 3 is ₹1,000 too high, but the skill says **Status: OK**

Test with: Quality cases + unit tests (Labs 1 and 2)
Fix: the instruction, script or knowledge

Passing one kind of test tells you nothing about the other. Ship only when both pass.

Rule 3 · Errors teach the model

When a tool fails, the model reads the error message and decides its next move. Write errors for the model, not for yourself.

The model calls the tool with: `{"lines": [...], "gst_rate": "18%"}`

Error that does not help

```
Traceback (most recent call last):
  ...
KeyError: 'gst_rate'
```

Model's next move: guesses, sends the same call again, or gives up.

Error that teaches

```
{"error": "gst_rate '18%' is not allowed",
 "fix": "Send gst_rate as one of
         0, 5, 12, 18, 28",
 "retryable": true}
```

Model's next move: sends gst_rate: 18 and the call succeeds.

`error`	what went wrong
`fix`	exactly what to send instead
`retryable`	can trying again help?

Check · Rules 2 and 3

CHECK

Q1 **The skill gives perfect summaries in tests, but in real chats it rarely loads. Which surface failed?**

A) Execution
B) Triggering
C) Permissions
D) Knowledge

Q2 **Which test catches "invoice 3 is ₹1,000 too high but the status says OK"?**

A) Trigger eval
B) Quality case / unit test
C) Security gate
D) check_skill

Q3 **Which error message follows Rule 3?**

A) Invalid input
B) Error 500
C) qty must be greater than 0. You sent -2.
D) Traceback … KeyError

Q4 **When should retryable be false?**

A) Wrong GST rate sent
B) A field is missing
C) Our own bug, or the HR system is down
D) An unknown field was sent

Check · Answers

ANSWERS

Q1 → B · Triggering

The job is done right when it runs, but it is not being chosen. Fix the description and test with trigger evals (Lab 3).

Q2 → B · Quality case / unit test

The skill loaded, but the answer was wrong: that is the execution surface (Labs 1 and 2).

Q3 → C · it says what to send instead

Only C tells the model how to fix its next call.

Q4 → C · retrying cannot help

A, B and D are fixable by the model. In C the same call will fail again, so the agent should stop and hand over to a human.

Rule 4 · Least privilege by default

No primitive gets network, write or credential access unless the manifest declares it. Default answer: deny.



Network

OFF unless declared



Write

OFF unless declared



Credentials

OFF unless declared



What could go wrong?

- A prompt-injected PDF tells the agent to email data out
- A script quietly writes over a shared file
- A token in env vars leaks into logs

If it is not declared, it cannot happen. The manifest turns 'we hope it is safe' into 'the gate checked it'.

Rule 5 · Two kinds of skill

Before you build a skill, ask one simple question:

Would ANY company want this exact skill, or only FIL?

Any company → Capability uplift

What it does	Teaches the model a technique it is still weak at
Think of it as	Extra coaching classes: useful until you can do it yourself
Examples	Read tables from scanned PDFs · Write correct SQL window functions
New AI model arrives	**Test the new model WITHOUT the skill. If it now does the job alone, retire the skill.**

Only FIL → Encoded preference

What it does	Tells the model how FIL wants a job done
Think of it as	FIL's employee handbook: always needed, whoever joins
Examples	Our invoice summary format · Month-end close checklist
New AI model arrives	**Keep the skill. Re-test that the new model still follows our rules.**

Why label it? The label tells us what to test when the AI model is upgraded.

Practice · Uplift or preference?

DISCUSS

In pairs, ask the question for each skill: any company, or only FIL? Then put its letter in a box.

A Write correct SQL window functions

B FIL expense-claim approval steps

C Read tables from scanned PDFs

D Our incident post-mortem format

E Summarise long legal contracts

F Quarterly risk report for FIL compliance

Uplift (any company)

? , ? , ?

Preference (only FIL)

? , ? , ?

Bonus: which ONE skill could stop being needed when a better AI model arrives? Why?

Practice · Answers

ANSWERS

Uplift (any company)

A SQL window functions: every company writes SQL

C Reading tables from scanned PDFs: a general technique

E Summarising legal contracts: a general technique

Preference (only FIL)

B FIL's own approval steps

D FIL's own post-mortem format

F FIL's own compliance report

Bonus answer: C. AI models are getting better at reading scanned tables every year. When a new model can do it well alone, this skill can be retired. B, D and F will never be learned by a public model: only FIL knows its own rules.

Lab 4 · Permission manifest + security gate

LAB

20 minutes · pairs · uses the provided gate.py

1. Add allowed-tools and the metadata fil.permissions block to your SKILL.md
2. Run: python gate.py invoice-summary/ and read the report
3. The gate scans scripts/ for network calls, file writes and env-var reads
4. For each finding: declare it in the manifest, or remove the code
5. Re-run until the gate prints PASS
6. Swap folders with a neighbour: can you find something the gate missed?

Done when…

- Gate prints PASS
- Every declared scope is actually used
- network_egress is 'none' unless justified
- Neighbour review done

Badge unlocked
Security Gatekeeper

Lab 4 · Make the skill safe to ship

Declare what the skill may touch, remove what it does not need, and find what the security gate missed.

Before you start

1. **Open a terminal in the Lab 4 student folder**

   The folder that contains invoice-summary-formatter/ tools/

2. **Install once**

   ```
   pip install -r tools/requirements.txt
   ```

3. **Run the security gate**

   ```
   python tools/gate.py invoice-summary-formatter
   ```

4. **You should see: RESULT: FAIL (7 problem(s))**

   That is correct: the skill is not safe yet.

5. **After EVERY step: [OK] or what to fix**

   ```
   python tools/check_lab4.py
   ```

Step	What you do	You should see
1	Read the gate report	7 problems
2	Add the manifest to SKILL.md	3 problems
3	Replace helpers.py	RESULT: PASS
4	Fix what the gate missed	[OK] Step 4
5	Neighbour review	Badge

Step 1 · Read the gate report

This is what you see now (real output):

```
[FAIL] write        scripts/export_summary.py:18  out.mkdir()
[FAIL] write        scripts/export_summary.py:19  (out / f'{safe}.md').write_text()
[FAIL] network      scripts/helpers.py:6          import requests
[FAIL] credentials  scripts/helpers.py:19         os.environ
[FAIL] write        scripts/helpers.py:20         open(AUDIT_LOG, 'a')
 - manifest: 'allowed-tools' is missing …   - manifest: 'metadata: fil.permissions' block is missing
```

Word	Means the code…
write	creates or changes a file
network	talks to the internet
credentials	reads a secret (password, token)

Each line shows: [FAIL] kind file:line the code it found

For every [FAIL] you choose:

A. Declare it in SKILL.md, if the skill really needs it

B. Remove the code, if it does not

When in doubt: remove.

Step 2 · Add the permission manifest

File: `invoice-summary-formatter/SKILL.md`

1 **Find the top of the file**

the line that starts with `description:` and the `---` line just below it

2 **Paste the block on the right**

between the description line and that `---` (no blank line, keep the spaces)

3 **Save, then run**

```
python tools/gate.py
   invoice-summary-formatter
```

RESULT: FAIL (3 problems)

Down from 7. The 2 output/ writes are now OK.

```
allowed-tools: Read Bash(python:scripts/*)
metadata:
  fil.permissions:
    data_scopes: [ap.invoices:read]
    network_egress: none
    writes: [./output/]
    credentials: none
    side_effects: none
```

Line	Says the skill may…
network_egress: none	not use the internet
writes: [./output/]	write files only inside output/
credentials: none	not read any secret

Step 3 · Declare or remove? Decide for each FAIL

The 3 problems left are all in helpers.py:

Gate says	What the code does	Does an invoice summary need it?	Decision
network import requests	Calls an internet currency API	No. FIL AP invoices are in INR	REMOVE
credentials os.environ	Reads ERP_SERVICE_TOKEN (a secret)	No. And it writes part of it into a log	REMOVE
write ../../shared/…	Writes into a team-wide shared folder	It needs a log, but not there	MOVE to output/



The wrong way to pass: write network_egress: [api.example-fx.com] and credentials: [ERP_SERVICE_TOKEN] in the manifest. The gate would pass, but the skill could now send data out and read a secret it never needed. check_lab4.py will refuse this.

Step 3 · Replace helpers.py

File: `invoice-summary-formatter/scripts/helpers.py`

1. **Select everything in the file**

   Ctrl+A, then Delete

2. **Paste the new file**

   on the right: no requests, no token, log inside output/

3. **Save, then run**

```
python tools/gate.py
   invoice-summary-formatter
```

RESULT: PASS

Then run check_lab4.py: it says Step 4 is still open.
The gate missed something.

```
"""Small helpers used by export_summary.py."""
import csv
import datetime

AUDIT_LOG = "output/audit_log.csv"


def log_run(invoice_no, status):
    """Append one line to this skill's own audit log."""
    row = [datetime.datetime.now().isoformat(), invoice_no, status]
    with open(AUDIT_LOG, "a", newline="") as f:
        csv.writer(f).writerow(row)
```

Step 4 · Fix what the gate missed

File: `invoice-summary-formatter/scripts/export_summary.py`

The gate PASSED, but this file still reads an environment variable. It hides it by renaming: from os import environ as env . The gate only looks for the name os.environ, so it did not see it.

Edit 1 · Delete this line (line 8):

```
from os import environ as env
```

Edit 2 · Find these 2 lines:

```
def main(invoice_no, status, summary_text):
    reviewer = env.get("AP_REVIEWER_EMAIL", "ap-review@fil.example")
```

Replace them with this 1 line:

```
def main(invoice_no, status, summary_text, reviewer="ap-review@fil.example"):
```

Save, then run: python tools/check_lab4.py → [OK] Step 4 … All steps done

Step 5 · Neighbour review

Swap folders with the pair next to you. Read every .py file in scripts/ and answer:

Does any file talk to the internet?

requests, httpx, urllib, socket, http …

Does any file read a secret?

environ, getenv, .env, token, password …

Does any file write outside output/?

open(… 'w' or 'a'), write_text, ../

Is anything renamed to hide it?

import … as … · from … import … as …

Done when

check_lab4.py prints All steps done

network_egress and credentials are none

your neighbour found nothing new (or you fixed it)

Badge unlocked

Security Gatekeeper

If something does not match

First run python tools/check_lab4.py : it names the step and the file to fix.

You see	Do this
Gate still says the fil.permissions block is missing	Check the spaces: 2 before fil.permissions, 4 before each key (Step 2)
Frontmatter is not valid YAML	A tab or a missing colon in the block: copy it again from the slide
[FAIL] network … import requests	helpers.py still has the old content: replace the whole file (Step 3)
[warn] … declared but no code uses it	Remove that line from the manifest: unused permission = extra risk
check_lab4 refuses network_egress or credentials	Set them back to none and remove the code instead
NameError: name 'env' is not defined	Step 4 edit 2 not done: the reviewer line still uses env
No module named 'yaml'	pip install -r tools/requirements.txt

Part E · The skill catalog taxonomy

Four kinds of skill, from one job to a whole role

Four kinds of skill



Atomic

One job

"Format an invoice per our template"



Composite

A procedure that calls other skills and tools

"Close the month"



Policy

Guardrails other skills must load

"PII handling", "Brand voice"

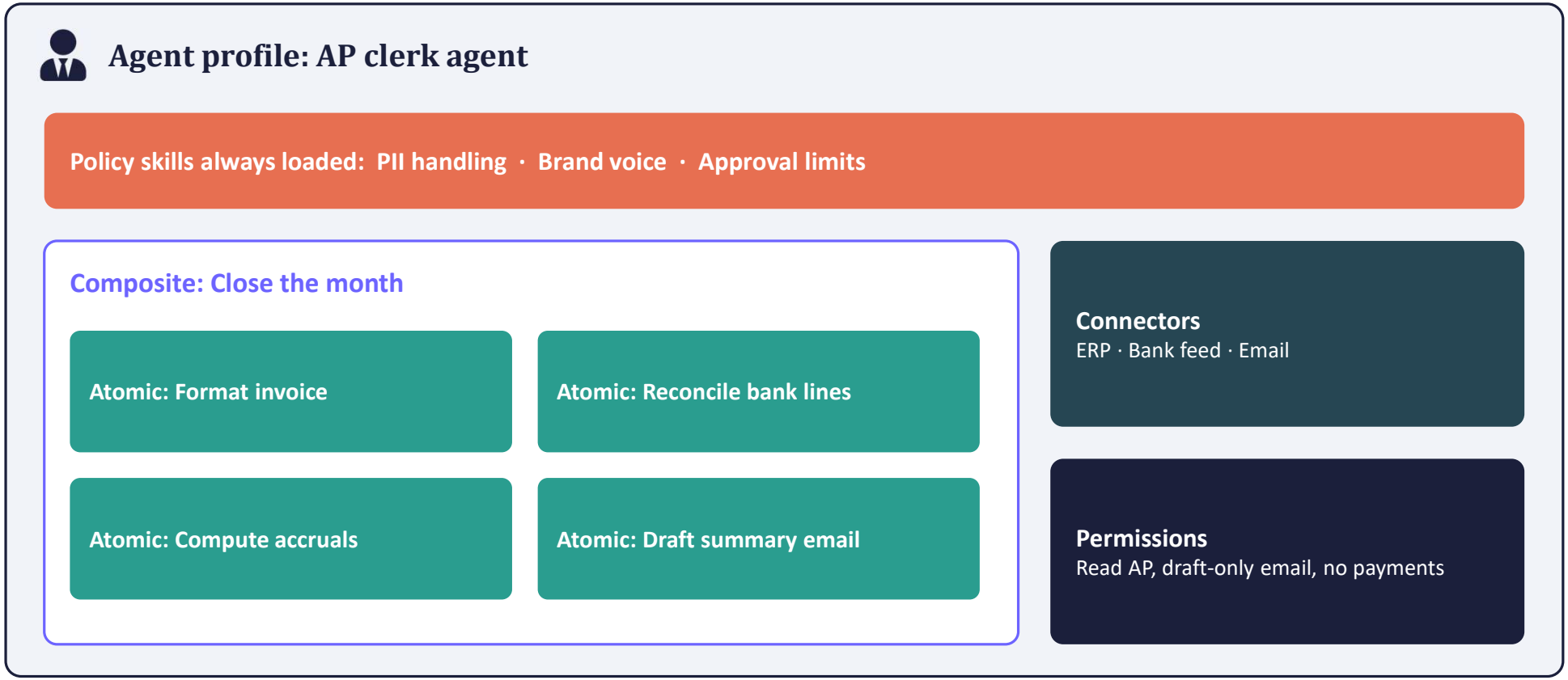


Agent profile

Named set of skills + connectors + permissions for a role

"AP clerk agent"

How they compose: the AP clerk agent



Check 3 · Sort the skill

CHECK

Q1 "Mask Aadhaar and PAN numbers before any output."

A) Atomic B) Composite C) Policy D) Agent profile

Q2 "Onboard a new vendor: verify GSTIN, create record, send welcome email."

A) Atomic B) Composite C) Policy D) Agent profile

Q3 "Convert an amount to words in lakh/crore format."

A) Atomic B) Composite C) Policy D) Agent profile

Q4 "Treasury analyst: cash skills + bank connector + read-only scopes."

A) Atomic B) Composite C) Policy D) Agent profile

Check 3 · Answers

ANSWERS

Q1 → C · Policy

A guardrail that other skills must load, regardless of what job they do.

Q2 → B · Composite

A multi-step procedure that calls other skills and tools in order.

Q3 → A · Atomic

One job, and a deterministic one: a strong candidate for a script.

Q4 → D · Agent profile

A named bundle of skills, connectors and permissions for a role.

Part F · Capstone lab

Assemble a complete Tier 1 skill, and review a peer's



Lab 5 · Capstone walkthrough

Three briefs, one recipe, solved step by step for classroom discussion

Lab 5 · Capstone: ship a complete skill

LAB

45 min build + 20 min review · teams of 3 · pick one brief

1	Pick a brief: A expense-claim-checker, B leave-balance-explainer, C meeting-minutes-formatter	
2	Build the Action first: script + JSON Schema contract + actionable errors	4
3	Write SKILL.md: trigger + all five instruction parts	1 · 2
4	Move rules to references/, the output template to assets/	3 · 5
5	Mock the connector as local JSON; declare the manifest; pass gate.py	6 · 7
6	Evals: 6 trigger prompts (3 + 3 near-misses) + 3 quality cases	8
7	Classify it (type + class), then demo in 2 minutes	

Grey tags = which of the 8 primitives the step builds

Done when…

- All 8 primitives present (connector mocked)
- check_skill + gate PASS
- Trigger score ≥ 5/6
- 3 quality cases match the contract
- Peer review ≥ 12/16

Badge unlocked
Tier 1 Skill Builder

Run of show and roles

PLAN

Time-box hard: a skill that runs beats a skill that is almost perfect

0–5	5–20	20–35	35–45	45–65	65+
Pick brief, assign roles	Action + SKILL.md + evals in parallel	Integrate: manifest, gate, first trigger run	Quality cases, fix, record scores	Peer review with the rubric	2-minute demos, badges

minutes

Domain author

Trigger, five instruction parts, references/, assets/

Engineer

Action script + schema, mock connector, manifest, gate

Tester

6 trigger prompts, 3 quality cases, runs every tool, keeps scores

One recipe, three skills

IDEA

The model does the language. The code does the rules.



Code decides (deterministic)

- Limits, dates, counts, money
- Who is allowed to see what
- The final Status value
- Errors the model can act on

Model decides (language)

- Reading messy user input
- Turning it into the JSON contract
- Explaining results in plain words
- Asking for what is missing

Three briefs side by side

DISCUSS

Discuss: which one would you pick, and where is the hardest part?

	A · expense-claim-checker	B · leave-balance-explainer	C · meeting-minutes-formatter
Who uses it	Managers approving claims	Every employee	Team leads
The trap	Per-day limits; receipts only above ₹500	Weekends and holidays; own balance only	'Someone' and 'by Sunday'
Action decides	Limits, tier, verdicts, approvable amount	Working days, casual rule, balance after	Owner attended? Date valid?
Model decides	Extract items, explain reasons	Resolve 'next Friday', suggest alternative	Decisions vs actions, wording
Connector (mocked)	HR · get_employee	HR · get_leave_balance (self)	Calendar + HR (two)
Statuses	APPROVE · PARTIAL · REJECT · NEEDS_INFO	ENOUGH · NOT_ENOUGH · NEEDS_INFO	COMPLETE · MISSING_OWNERS · NEEDS_INFO



Brief A · expense-claim-checker

FIL managers who approve travel expense claims. Pain today: Finance rejects about 1 in 5 approved claims later.

A · Step 1 · Read the brief as a contract

BRIEF A

expense-claim-checker: before writing anything, list what triggers it, what must not, and what it promises

What users type

- "Check Aarav's Bengaluru claim before I approve it"
- "How much of this hotel and cab claim is reimbursed?"
- "Which items in my Mumbai trip will get rejected?"

Must stay quiet

- Booking travel
- Applying for leave
- Payroll or salary questions
- Vendor invoices

Output promise

Claimed · Approvable · Not reimbursable · reason per item

`APPROVE`

`PARTIAL`

`REJECT`

`NEEDS_INFO`

The trap Limits are per day per category: two meals on the same day are added together. A receipt is needed only above ₹500.

A · Step 2 · SKILL.md: trigger + five parts

BRIEF A

The description is code: the router reads only this line before loading the skill

SKILL.md · frontmatter

```
---
name: expense-claim-checker
description: >
  Use when an employee or manager shares
  a domestic travel expense claim
  (hotel, meals, cabs) and asks to check
  it against the FIL expense policy,
  find what is reimbursable, or prepare
  it for approval. Do not use for
  booking travel, applying for leave,
  payroll or salary questions, or
  checking vendor invoices.
---
```

1. **Role**
   Expense-audit assistant for approving managers

2. **Procedure**
   Extract items → dates as YYYY-MM-DD → pipe JSON to policy_check.py → follow any fix (max 2) → fill template

3. **Decision rules**
   Status is the script's status · unknown item → 'other' · no ID → NEEDS_INFO · never show others' HR data

4. **Output contract**
   5-row table + items needing attention + Status + Notes, nothing else

5. **Examples**
   L1, Pune, hotel ₹4,300 → approvable ₹4,000 → PARTIAL

A · Step 3 · Knowledge, Asset, Connector

BRIEF A

Read to decide → references/ · used to produce → assets/ · live data → connector

Knowledge · 3

`references/expense_policy.md`

- Limits by grade: L1 meals 600, hotel 4,000 / 3,000
- Tier-1 cities: Mumbai, Delhi NCR, Bengaluru…
- Receipt above ₹500 · claim within 30 days
- Never: alcohol, fines, entertainment

Asset · 5

`assets/claim_summary_template.md`

```
| Employee | <name> (<id>) |
| Claimed | INR <amount> |
| Approvable | INR <amount> |
Status: <APPROVE | …>
Notes: <one sentence>
```

Connector · 6

In the lab (mock)
`mock_connectors/hr_employees.json`

In production
HR MCP server · get_employee

Scope
`hr.employees:read`

A · Step 4 · Action + JSON Schema

BRIEF A

The schema rejects bad input by itself; the error tells the model exactly what to send next

scripts/ · schema (JSON Schema 2020-12)

```
"employee_id": {"pattern": "^FIL[0-9]{4}$"},
"submitted_on": {"pattern": "YYYY-MM-DD"},
"items": {"minItems": 1, "items": {
  "category": {"enum": ["meals", "hotel",
               "local_travel", "other"]},
  "amount": {"exclusiveMinimum": 0},
  "receipt": {"type": "boolean"} }},
"additionalProperties": false
```

Bad input: "category": "laptop"

```
"error": "items[0].category = 'laptop'
          is not allowed",
"fix": "Use one of meals, hotel,
  local_travel, other. Anything else
  goes in 'other'.",
"retryable": true
```

Code owns Limits, tiers, per-day grouping, Decimal money, status

Model owns Reading messy claims, plain-language reasons

A · Steps 5–6 · Manifest, gate, evals

BRIEF A

Least privilege: read-only scopes, no network, no writes, no credentials

SKILL.md · permission manifest (7)

```
allowed-tools: Read Bash(python:scripts/*)
fil.permissions:
  data_scopes: [hr.employees:read,
                expense.claims:read]
  network_egress: none
  writes: none
  credentials: none
  side_effects: none
```

```
$ gate.py <skill>   RESULT: PASS
```

16/16

check_skill.py

PASS

gate.py

6/6

trigger evals

9/9

Action unit tests

A · Worked quality cases + classify

BRIEF A

3 cases cover the three outcomes: the happy path, a problem found, and missing information

Case	What the Action finds	Status
Neha · L2 · Mumbai · ₹8,270	All within L2 limits; ₹420 lunch without receipt is fine (≤ ₹500)	`APPROVE`
Aarav · L1 · Bengaluru · ₹7,550	Hotel 5,200 → 4,000 · meals 750 → 600 · beer ₹900 not allowed · auto ₹700 no receipt → ₹4,600	`PARTIAL`
FIL9999 · Jaipur · ₹3,300	Employee not in HR records; nothing guessed	`NEEDS_INFO`

Type: Atomic **Class: Encoded preference**

One job, FIL-specific policy. Re-test when the policy changes, not only on model upgrades.

Discuss Aarav's two meals were ₹450 and ₹300, each under ₹600. Why is ₹150 still cut?



Brief B · leave-balance-explainer

Every FIL employee. Pain today: HR answers about 200 'how much leave do I have?' messages a month.

B · Step 1 · Read the brief as a contract

BRIEF B

leave-balance-explainer: before writing anything, list what triggers it, what must not, and what it promises

What users type

- "How many casual leaves do I have left this year?"
- "Do I have enough earned leave for 10 to 14 November?"
- "Check my sick leave balance, I may need 3 days"

Must stay quiet

- Applying for or cancelling leave
- Payroll or salary questions
- Booking travel
- HR policy not about own balance

Output promise

Working days · balance now · balance after

`ENOUGH`

`NOT_ENOUGH`

`NEEDS_INFO`

The trap '5 days from 10 to 14 Nov' is 4 working days: 14 Nov 2026 is a Saturday. Let code count days, and only ever show the user's own balance.

B · Step 2 · SKILL.md: trigger + five parts

BRIEF B

The description is code: the router reads only this line before loading the skill

SKILL.md · frontmatter

```
---
name: leave-balance-explainer
description: >
  Use when an employee wants to check
  their own leave balance (casual, sick
  or earned leave left), or asks whether
  they have enough leave days for
  specific dates. Do not use to apply
  for, submit or cancel leave, for
  payroll or salary questions, booking
  travel, or HR policy questions not
  about the user's own balance.
---
```

1. **Role**
   Leave-balance helper for one employee's own question

2. **Procedure**
   ID first (never guess) → type + dates, resolve 'next Friday' → pipe JSON to check_leave.py → fill template

3. **Decision rules**
   Script's status · explain DAYS_CORRECTED · NOT_ENOUGH → one alternative · other person → NEEDS_INFO

4. **Output contract**
   6-row table + Status + Notes, nothing else

5. **Examples**
   FIL2088, sick 12–13 Nov → 2 days, 8 → 6 → ENOUGH; no ID → NEEDS_INFO

B · Step 3 · Knowledge, Asset, Connector

BRIEF B

Read to decide → references/ · used to produce → assets/ · live data → connector

Knowledge · 3

`references/leave_rules.md`

- Working days: Mon–Fri minus FIL holidays
- Holidays: 2026-11-09, 2026-11-24, 2026-12-25
- Casual: at most 3 consecutive days
- Earned: half day allowed · PL means earned

Asset · 5

`assets/leave_answer_template.md`

```
| Leave type | <type> |
| Working days | <n> |
| Balance now | <n> |
| Balance after | <n> |
Status: <ENOUGH | …>
```

Connector · 6

In the lab (mock)
`mock_connectors/hr_employees.json`

In production
HR MCP · get_leave_balance (on behalf of the user)

Scope
`hr.leave_balance:read:self`

B · Step 4 · Action + JSON Schema

BRIEF B

The schema rejects bad input by itself; the error tells the model exactly what to send next

scripts/ · schema (JSON Schema 2020-12)

```
"required": ["employee_id", "leave_type",
             "start_date", "end_date"],
"leave_type": {"enum": ["casual", "sick",
                        "earned"]},
"start_date": {"pattern": "YYYY-MM-DD"},
"half_day": {"type": "boolean"},
"days_user_said": {"minimum": 0},
"additionalProperties": false
```

Bad input: employee_id missing

```
"error": "employee_id is missing",
"fix": "Do not guess an ID. Ask the
  user for their employee ID
  (like FIL1023), then call again.",
"retryable": false
```

Code owns Working days, weekends, holidays, casual 3-day rule, balance maths

Model owns Resolving 'next Friday', a polite correction, one alternative

B · Steps 5–6 · Manifest, gate, evals

BRIEF B

Least privilege: read-only scopes, no network, no writes, no credentials

SKILL.md · permission manifest (7)

```
allowed-tools: Read Bash(python:scripts/*)
fil.permissions:
  data_scopes:
    [hr.leave_balance:read:self]
  network_egress: none
  writes: none
  credentials: none
  side_effects: none
```

```
$ gate.py <skill>   RESULT: PASS
```

16/16

check_skill.py

PASS

gate.py

4 → 6/6

trigger evals (tuned)

11/11

Action unit tests

B · Worked quality cases + classify

BRIEF B

3 cases cover the three outcomes: the happy path, a problem found, and missing information

Case	What the Action finds	Status
Aarav · earned · 10–14 Nov	4 working days (Sat 14 Nov excluded), user said 5 → DAYS_CORRECTED; 11.5 → 7.5	`ENOUGH`
Priya · casual · 30 Oct + 2 Nov	2 working days (weekend excluded); casual balance 0 → suggest earned (2 left)	`NOT_ENOUGH`
No employee ID · sick · 3 days	Script refuses with retryable: false; skill asks for the ID	`NEEDS_INFO`

Type: Atomic **Class: Encoded preference**

One job, FIL holidays and leave rules. Re-test every January when the holiday list changes.

Discuss A manager asks: 'How much leave does Priya have?' What should the skill do, and which line stops it?



Brief C · meeting-minutes-formatter

FIL team leads. Pain today: Minutes are free text; action items get lost because nobody owns them.

C · Step 1 · Read the brief as a contract

BRIEF C

meeting-minutes-formatter: before writing anything, list what triggers it, what must not, and what it promises

What users type

- "Turn my notes from today's sprint review into proper minutes"
- "Write up the minutes for EVT-7781 from these notes"
- "Format these notes and list the action items with owners"

Must stay quiet

- Scheduling or rescheduling meetings
- Agendas for future meetings
- Summarising non-meeting documents
- Sending emails

Output promise

Attendees · decisions · actions with owner and due date

`COMPLETE`

`MISSING_OWNERS`

`NEEDS_INFO`

The trap 'Someone' is not an owner and 'by Sunday' is not a date. The model must send null, never invent a person or a date.

C · Step 2 · SKILL.md: trigger + five parts

BRIEF C

The description is code: the router reads only this line before loading the skill

SKILL.md · frontmatter

```
---
name: meeting-minutes-formatter
description: >
  Use when a team lead shares rough
  notes from a meeting that already
  happened and asks to turn them into
  proper minutes, write up the minutes,
  or list the decisions and action items
  with owners. Do not use to schedule,
  reschedule or cancel a meeting, find a
  free slot, write an agenda for a
  future meeting, summarise documents
  that are not meetings, or send emails.
---
```

1. **Role**
   Minutes writer who makes sure every action has an owner and a date
2. **Procedure**
   Event ID first → split decisions / actions → owner ID or null, date or null → pipe to check_actions.py → fill template
3. **Decision rules**
   Status is the script's · drop 'off the record' remarks · owner not an attendee → keep and flag · list fixes by number
4. **Output contract**
   Heading, attendees, decisions, action table with Check column + Status + Notes
5. **Examples**
   'someone to book the demo room' → — · — · NO_OWNER → MISSING_OWNERS

C · Step 3 · Knowledge, Asset, Connector

BRIEF C

Read to decide → references/ · used to produce → assets/ · live data → connector

Knowledge · 3

`references/minutes_rules.md`

- Decision = agreed; action = person + date
- Owner must have attended the meeting
- Due date on or after the meeting date
- Never: personal remarks, 'off the record'

Asset · 5

`assets/minutes_template.md`

```
# Minutes: <title>
**Attendees:** <names>
## Decisions
| Task | Owner | Due |
Status: <COMPLETE | …>
```

Connector · 6

In the lab (mock)
`calendar_events.json` + `hr_employees.json`

In production
Calendar MCP · get_event + HR MCP · get_employee

Scope
`calendar.events:read`, `hr.employees:read`

C · Step 4 · Action + JSON Schema

BRIEF C

The schema rejects bad input by itself; the error tells the model exactly what to send next

scripts/ · schema (JSON Schema 2020-12)

```
"event_id": {"pattern": "^EVT-[0-9]{4}$"},
"actions": {"items": {
  "required": ["task", "owner", "due"],
  "owner": {"type": ["string", "null"],
            "pattern": "^FIL[0-9]{4}$"},
  "due":   {"type": ["string", "null"],
            "pattern": "YYYY-MM-DD"} }},
"additionalProperties": false
```

Bad input: "owner": "someone", "due": "Sunday"

```
"error": "actions[0].due = 'Sunday'
          has the wrong format",
"fix": "owner is an employee ID like
  FIL2088 (or null); due is YYYY-MM-DD
  (or null). Never send 'someone'."
```

Code owns Owner attended? Date valid and after the meeting? Names, status

Model owns Decisions vs actions, short task wording, leaving out remarks

C · Steps 5–6 · Manifest, gate, evals

BRIEF C

Least privilege: read-only scopes, no network, no writes, no credentials

SKILL.md · permission manifest (7)

```
allowed-tools: Read Bash(python:scripts/*)
fil.permissions:
  data_scopes: [calendar.events:read,
                hr.employees:read]
  network_egress: none
  writes: none
  credentials: none
  side_effects: none
```

```
$ gate.py <skill>    RESULT: PASS
```

16/16

check_skill.py

PASS

gate.py

5 → 6/6

trigger evals (tuned)

10/10

Action unit tests

C · Worked quality cases + classify

BRIEF C

3 cases cover the three outcomes: the happy path, a problem found, and missing information

Case	What the Action finds	Status
EVT-7781 · sprint review	3 actions, each owner attended, each date after 21 Oct	`COMPLETE`
EVT-7790 · capstone planning	'prepare 3 cases by Sunday' and 'someone to book the room' → NO_OWNER; Priya's action OK	`MISSING_OWNERS`
Vendor sync, no event ID	Attendees cannot be checked; skill asks for the event ID	`NEEDS_INFO`

Type: Atomic **Class: Encoded preference**

The FIL minutes standard is a house rule. If it also emails the minutes it becomes composite and gains a side effect.

Discuss Why does the contract allow owner: null instead of making owner required text?

1 2 3 4 5 6 7 8

Next: Tier 2 · Skills OS

How skills are discovered, routed, versioned and run at scale, built entirely from today's blocks

Questions?